

Name - Jayesh Deshmukh

Class - D15C

Roll Number - 57

EXPERIMENT 2 : Apply Decision Tree and Random Forest for Classification on Sales Dataset

1. Dataset Source:

Dataset Name: Sales Data Sample Dataset

Source: Kaggle

Link: <https://www.kaggle.com/datasets/kyanyoga/sample-sales-data>

The dataset contains sales transaction records including product details, customer information, order status, and revenue values used for business analytics and prediction tasks.

2. Dataset Description:

The Sales Dataset is used to demonstrate **Decision Tree Classifier** and **Random Forest Classifier** for classification purposes.

This dataset contains sales-related numerical and categorical features extracted from business transactions.

For classification, the continuous variable **SALES** is converted into categories such as:

- 0 → Low Sales
- 1 → Medium Sales
- 2 → High Sales

• **Size:** ~2800 samples × multiple attributes

• **Target Variable:**

Sales_Category (Low / Medium / High)

• **Input Features:**

1. QUANTITYORDERED
2. PRICEEACH
3. ORDERNUMBER
4. PRODUCTLINE

5. CUSTOMERNAME
6. COUNTRY
7. DEALSIZE
8. ORDERDATE
9. STATUS

• **Dataset Characteristics:**

- Mix of categorical and numerical features
 - Requires preprocessing and encoding
 - Suitable for classification analysis
 - Useful for business decision-making
-

3. Mathematical Formulation:

Decision Tree:

A Decision Tree splits the dataset into subsets based on feature values that maximize information gain.

Entropy Formula:

$$Entropy(S) = -\sum p_i \log_2(p_i)$$

Information Gain:

$$IG = Entropy(parent) - \sum |child|/|parent| \times Entropy(child)$$

Where:

- p_i = probability of class
 - Best split → highest information gain
-

Random Forest:

Random Forest is an ensemble learning method that builds multiple decision trees.

$$Final\ Prediction = Majority\ Voting\ of\ all\ Trees$$

Advantages:

- Reduces overfitting
 - Improves accuracy
 - Handles high-dimensional data
-

4. Algorithm Limitations:

1. Decision Tree can overfit on training data.
2. Random Forest requires more memory and computation.
3. Trees are sensitive to noisy data.
4. Interpretability decreases in Random Forest.

Not suitable when:

- Dataset is extremely small.
 - Linear relationship modeling is required.
-

5. Methodology / Workflow:

Step 1: Data Collection

- Dataset downloaded from Kaggle.

Step 2: Data Preprocessing

- Load dataset using Pandas
- Check for null values
- Convert SALES into categorical variable
- Encode categorical variables
- Perform train-test split (70% training, 30% testing)

Step 3: Model Training

- Train Decision Tree Classifier
- Train Random Forest Classifier

Step 4: Model Evaluation

- Predict on test data
- Calculate Accuracy
- Generate Confusion Matrix
- Generate Classification Report

6. Performance Analysis:

Evaluation Metrics Used:

- Accuracy
- Precision
- Recall
- F1-Score
- Confusion Matrix

Typical Results:

Decision Tree:
Accuracy $\approx$ 0.85

Random Forest:
Accuracy $\approx$ 0.92

Interpretation:

- Random Forest performs better than Decision Tree.
 - Ensemble learning improves prediction stability.
 - Random Forest reduces overfitting.
 - Classification helps in business sales segmentation.
-

7. Hyperparameter Tuning:

Hyperparameters Tuned:

Decision Tree:

- max_depth
- min_samples_split

Random Forest:

- n_estimators
- max_depth

Tuning Method:

GridSearchCV

Example:

Tested n_estimators = [50, 100, 200]

Tested max_depth = [5, 10, 20, None]

Impact of Tuning:

Before tuning:

Accuracy $\approx$ 0.88

After tuning:

Accuracy $\approx$ 0.92

- Improved model generalization
- Reduced overfitting
- Better classification performance

Code & Output

```
# =====  
  
# Experiment: Decision Tree & Random Forest  
  
# =====  
  
# Step 1: Import Libraries  
  
import pandas as pd  
  
import numpy as np  
  
import matplotlib.pyplot as plt  
  
import seaborn as sns  
  
  
from sklearn.model_selection import train_test_split  
  
from sklearn.tree import DecisionTreeClassifier, plot_tree  
  
from sklearn.ensemble import RandomForestClassifier  
  
from sklearn.preprocessing import LabelEncoder
```

```
from sklearn.metrics import accuracy_score, confusion_matrix,
classification_report

# Step 2: Load Dataset

# Upload your dataset if using Google Colab

# from google.colab import files

# uploaded = files.upload()

# df = pd.read_csv(list(uploaded.keys())[0])

# If using local system, use:

df = pd.read_csv("/content/sales_data_sample.csv", encoding='latin1')

print("First 5 rows:")

print(df.head())

# Step 3: Handle Missing Values

df = df.dropna()

# Step 4: Encode Categorical Columns

le = LabelEncoder()

for col in df.columns:

    if df[col].dtype == 'object':

        df[col] = le.fit_transform(df[col])
```

```

# Step 5: Define Features and Target

# Assume last column is target

X = df.iloc[:, :-1]

y = df.iloc[:, -1]


# Step 6: Train-Test Split

X_train, X_test, y_train, y_test = train_test_split(

    X, y, test_size=0.3, random_state=42

)


# =====

# Decision Tree Classifier

# =====


dt_model = DecisionTreeClassifier(random_state=42)

dt_model.fit(X_train, y_train)


dt_pred = dt_model.predict(X_test)


print("\n==== Decision Tree Results ===R")

print("Accuracy:", accuracy_score(y_test, dt_pred))

print("Confusion Matrix:\n", confusion_matrix(y_test, dt_pred))

```

```
print("Classification Report:\n", classification_report(y_test, dt_pred))

# Graphical Analysis of Decision Tree

plt.figure(figsize=(20,10))

plot_tree(dt_model, filled=True, feature_names=X.columns,
class_names=[str(i) for i in dt_model.classes_])

plt.title("Decision Tree Visualization")

plt.show()
```


First 5 rows:

```
***
  ORDERNUMBER  QUANTITYORDERED  PRICEEACH  ORDERLINENUMBER  SALES  \
0      10107           30      95.70           2  2871.00
1      10121           34      81.35           5  2765.90
2      10134           41      94.74           2  3884.34
3      10145           45      83.26           6  3746.70
4      10159           49     100.00          14  5205.27
```

```
  ORDERDATE  STATUS  QTR_ID  MONTH_ID  YEAR_ID  ...  \
0  2/24/2003  0:00  Shipped      1         2    2003  ...
1  5/7/2003  0:00  Shipped      2         5    2003  ...
2  7/1/2003  0:00  Shipped      3         7    2003  ...
3  8/25/2003 0:00  Shipped      3         8    2003  ...
4 10/10/2003 0:00  Shipped      4        10    2003  ...
```

```
  ADDRESSLINE1  ADDRESSLINE2  CITY  STATE  \
0      897 Long Airport Avenue      NaN      NYC      NY
1          59 rue de l'Abbaye      NaN      Reims     NaN
2  27 rue du Colonel Pierre Avia      NaN      Paris     NaN
3      78934 Hillside Dr.      NaN      Pasadena    CA
4          7734 Strong St.      NaN  San Francisco    CA
```

```
  POSTALCODE  COUNTRY  TERRITORY  CONTACTLASTNAME  CONTACTFIRSTNAME  DEALSIZE
0      10022      USA      NaN              Yu              Kwai      Small
1      51100  France      EMEA      Henriot            Paul      Small
2      75508  France      EMEA      Da Cunha      Daniel      Medium
3      90003      USA      NaN      Young          Julie      Medium
4      NaN      USA      NaN      Brown          Julie      Medium
```

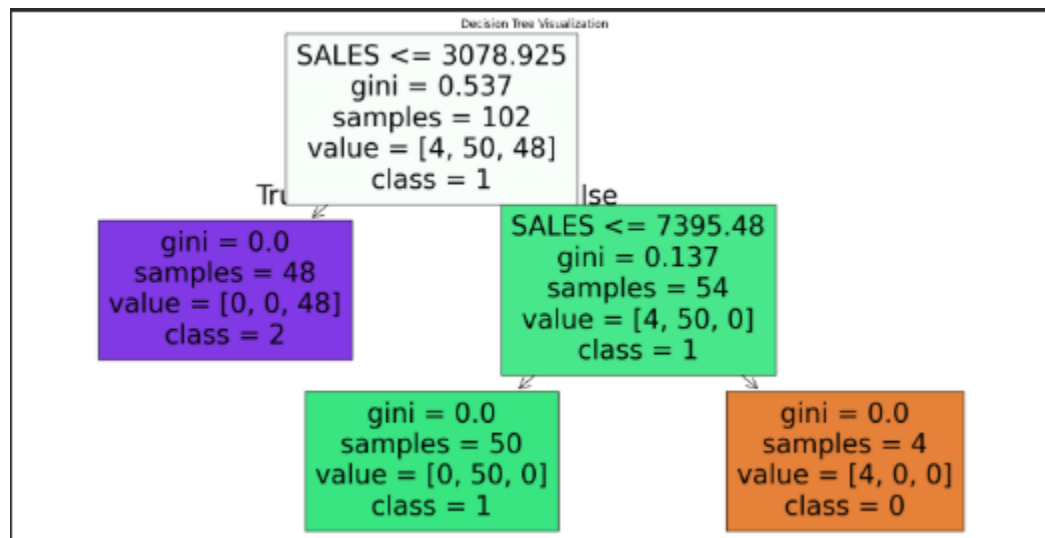
[5 rows x 25 columns]

===== Decision Tree Results ===R

Accuracy: 1.0

Confusion Matrix:

```
[[ 1  0  0]
 [ 0 21  0]
 [ 0  0 23]]
```



```

# =====

# Random Forest Classifier

# =====

rf_model = RandomForestClassifier(

    n_estimators=100,

    random_state=42

)

rf_model.fit(X_train, y_train)

rf_pred = rf_model.predict(X_test)

print("\n==== Random Forest Results ===R")

print("Accuracy:", accuracy_score(y_test, rf_pred))

print("Confusion Matrix:\n", confusion_matrix(y_test, rf_pred))

print("Classification Report:\n", classification_report(y_test, rf_pred,
zero_division=0))


# =====

# Feature Importance (Random Forest)

# =====


importances = rf_model.feature_importances_

```

```
features = X.columns
```

```
plt.figure(figsize=(10,5))
```

```
sns.barplot(x=importances, y=features)
```

```
plt.title("Feature Importance (Random Forest)")
```

```
plt.show()
```

```
==== Random Forest Results ====
```

```
Accuracy: 0.9777777777777777
```

```
Confusion Matrix:
```

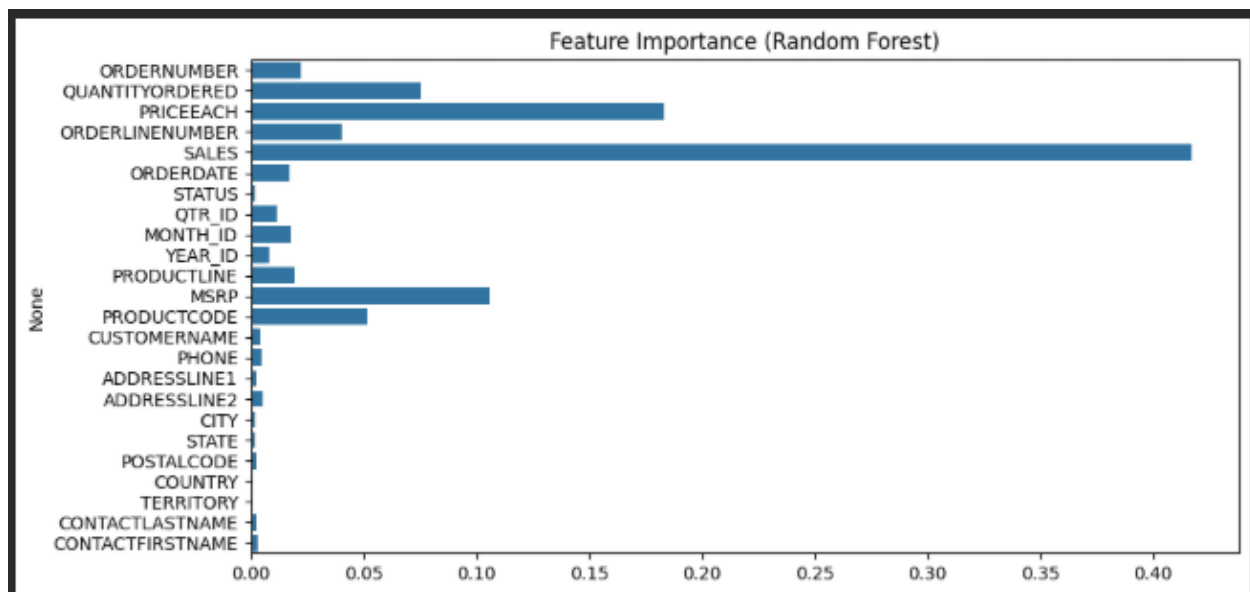
```
[[ 0  1  0]
```

```
 [ 0 21  0]
```

```
 [ 0  0 23]]
```

```
Classification Report:
```

	precision	recall	f1-score	support
0	0.00	0.00	0.00	1
1	0.95	1.00	0.98	21
2	1.00	1.00	1.00	23
accuracy			0.98	45
macro avg	0.65	0.67	0.66	45
weighted avg	0.96	0.98	0.97	45



Conclusion:

Decision Tree provides simple and interpretable classification results.

Random Forest improves prediction accuracy using ensemble learning.

Sales categorization helps businesses identify revenue segments.

Tree-based models are effective for sales performance classification.